EDD MedTrack

Fase 2 — Sistema Hospitalario

Manual Técnico

Estructuras de Datos — Universidad San Carlos de Guatemala

Facultad de Ingeniería — Ingeniería en Ciencias y Sistemas

Arquitectura modular del sistema EDD MedTrack Fase 2:
estructuras de datos, interfaz GTK/Perl, pipeline de validación JSON,
generación de reportes Graphviz y persistencia de datos.

Autor: Angel Raúl Herrera Chilel
Carnet: 202402587
Sección: B
Proyecto: EDD_Proyecto1_Fase2
Fecha: 8 de abril de 2026

Índice

Introducción **3**

1 Requerimientos del Sistema **4**
1.1 Entorno de Desarrollo . . . 4
1.2 Requisitos Mínimos para Ejecución . . . 4

2 Estructuras de Datos **4**
2.1 Árbol AVL — `gestion_datos/avl.pm` . . . 4
2.1.1 Nodo AVL . . . 4
2.1.2 Operaciones . . . 5
2.1.3 Rotaciones . . . 5
2.2 Árbol BST — `gestion_datos/bst.pm` . . . 5
2.2.1 Nodo BST . . . 5
2.2.2 Eliminación en BST . . . 5
2.3 Árbol B Orden 4 — `gestion_datos/btree_orden4.pm` . . . 5
2.3.1 Propiedades . . . 5
2.3.2 Inserción y división de nodos . . . 6
2.3.3 Eliminación . . . 6
2.4 Lista Doblemente Enlazada — `gestion_datos/lista_doblemente_enlazada.pm` 6
2.5 Lista Circular Doblemente Enlazada — `gestion_datos/lista_circular_doblemente_enlazada.`
2.6 Matriz Dispersa — `gestion_datos/matriz_dispersa.pm` . . . 6

3 Módulo de Validación JSON **6**
3.1 `utilidades/validar_json.pm` . . . 6
3.1.1 `normalizar_utf8($texto)` . . . 6
3.1.2 `limpiar_campo($valor, $campo, $tipo)` . . . 7
3.1.3 `validar_campos_obligatorios($item, $campos)` . . . 7

4 Pipeline de Carga Masiva **7**
4.1 `utilidades/carga_masiva_inventario.pm` . . . 7
4.1.1 Flujo de `procesar_archivo($ruta, $ctx)` . . . 7
4.2 `utilidades/carga_masiva_usuarios.pm` . . . 7

5 Interfaz Gráfica GTK3 **8**
5.1 Arquitectura . . . 8
5.1.1 Contexto compartido (`%ctx`) . . . 8
5.1.2 `interfaz/admin/common.pm` . . . 8
5.1.3 `interfaz/admin/personal_medico.pm` . . . 8

6 Generación de Reportes Graphviz **9**
6.1 Flujo general . . . 9
6.2 Características por reporte . . . 9

7 Persistencia JSON **9**
7.1 `utilidades/leer_json.pm` . . . 9

8 Módulo de Solicitudes y Pedidos **10**
8.1 `interfaz/admin/solicitudes.pm` . . . 10
8.1.1 Estructura de archivos de solicitudes . . . 10

9 Sistema de Estilos GTK3 **11**

9.1 `interfaz/estilos.pm` 11
9.1.1 Widgets y selectores críticos 11

10 Manejo de Errores **11**

11 Conclusiones **11**

Introducción

Este manual técnico describe la arquitectura interna del sistema **EDD MedTrack Fase 2**, desarrollado íntegramente en Perl con interfaz gráfica GTK3. El sistema implementa cinco estructuras de datos desde cero (sin librerías externas): Árbol AVL, Árbol BST, Árbol B de Orden 4, Lista Doblemente Enlazada y Lista Circular Doblemente Enlazada, más una Matriz Dispersa para análisis de inventario por proveedor/fabricante.

El documento está dirigido a desarrolladores que necesiten comprender, mantener o extender el código fuente, su organización de paquetes y la interacción entre módulos.

Requerimientos del Sistema

Entorno de Desarrollo

Componente	Especificación
Dispositivo	Laptop NIAKUN — Intel Core i9-11900H, 16 GB RAM, NVMe 1 TB
Sistema Operativo	Arch Linux (rolling release)
Versión de Perl	v5.38 (Arch Linux)
Editor	Neovim
Interfaz gráfica	GTK3 vía `Gtk3` Perl bindings
Reportes	Graphviz (comando `dot`)
Módulos Perl	`JSON::PP`, `Encode`, `Time::Piece`, `Glib`

Requisitos Mínimos para Ejecución

Componente	Mínimo
Sistema Operativo	Linux / macOS / Windows con WSL
Perl	v5.30 o superior
GTK3	`libgtk-3` + bindings Perl (`Gtk3`, `Glib`)
Graphviz	Paquete `graphviz` con el comando `dot` en PATH
Memoria RAM	4 GB

Estructuras de Datos

Todas las estructuras están implementadas desde cero en Perl sin librerías externas. Cada módulo expone una interfaz pública consistente: `nuevo()`, `insertar()`, `buscar()`, `eliminar()`, `en_orden()`, y métodos de serialización.

Árbol AVL — `gestion_datos/avl.pm`

Almacena el personal médico ordenado por número de colegio profesional.

Nodo AVL

Cada nodo contiene: `colegio` (clave), `nombre`, `tipo`, `departamento`, `especialidad`, `contrasena`, `altura`, `izq` y `der` (referencias a hijos).

Operaciones

Método	Descripción
`insertar(%datos)`	Inserta un nodo y rebalancea con rotaciones simples/dobles
`buscar($colegio)`	Búsqueda O(log n) por clave
`eliminar($colegio)`	Eliminación con sucesor in-orden y rebalanceo
`en_orden($cb)`	Recorrido in-orden, llama al callback con cada nodo
`pre_orden($cb)`	Recorrido pre-orden
`post_orden($cb)`	Recorrido post-orden
`a_array_json()`	Serializa a arrayref para guardar en JSON
`cargar_desde_array($arr)`	Reconstruye el árbol desde un arrayref

Rotaciones

El árbol calcula el **factor de balance** = altura(izq) - altura(der). Si el factor es > 1 o < -1, se aplica la rotación correspondiente:

Caso	Condición	Rotación
Izquierda-Izquierda	balance > 1, balance(izq) ≥ 0	Simple derecha
Izquierda-Derecha	balance > 1, balance(izq) < 0	Doble: izq luego der
Derecha-Derecha	balance < -1, balance(der) ≤ 0	Simple izquierda
Derecha-Izquierda	balance < -1, balance(der) > 0	Doble: der luego izq

Árbol BST — `gestion_datos/bst.pm`

Almacena el inventario de equipos médicos (prefijo `EQU-`) con búsqueda O(log n) en el caso promedio.

Nodo BST

Contiene: `codigo` (clave), `nombre`, `fabricante`, `precio_unitario`, `cantidad`, `fecha_ingreso`, `nivel_minimo`, `izq` y `der`.

Eliminación en BST

La eliminación de un nodo con dos hijos utiliza el sucesor in-orden (mínimo del subárbol derecho): el valor del sucesor reemplaza al nodo eliminado y el sucesor se elimina del subárbol derecho. No se realizan rotaciones (a diferencia del AVL).

Árbol B Orden 4 — `gestion_datos/btree_orden4.pm`

Almacena el inventario de suministros médicos (prefijo `SUM-`). Cada nodo puede contener hasta **3 claves** y **4 hijos**.

Propiedades

- Orden mínimo: todo nodo no raíz tiene al menos $\lceil 4/2 \rceil - 1 = 1$ clave.
- Orden máximo: todo nodo tiene como máximo 3 claves.
- Todas las hojas están en el mismo nivel.
- Las claves de cada nodo están ordenadas de forma ascendente.

Inserción y división de nodos

Al insertar en un nodo lleno (3 claves), se realiza un **split**: la clave media asciende al nodo padre y el nodo se divide en dos. Si el padre también está lleno, el proceso se propaga hacia arriba recursivamente. Si la raíz se divide, se crea una nueva raíz incrementando la altura del árbol.

Eliminación

Maneja tres casos: eliminación de clave en hoja, eliminación de clave en nodo interno (reemplazar con predecesor/sucesor de hoja) y fusión o redistribución de nodos cuando un nodo queda con menos de la cantidad mínima de claves.

Lista Doblemente Enlazada — `gestion_datos/lista_doblemente_enlazada.pm`

Almacena el inventario de medicamentos (prefijo `MED-`).

Cada nodo contiene: `codigo`, `nombre`, `principio_activo`, `laboratorio`, `fabricante`, `precio_unitario` `cantidad`, `fecha_vencimiento`, `nivel_minimo`, más punteros `anterior` y `siguiente`.

La inserción mantiene el orden ascendente por código. El método `buscar_por_codigo($codigo)` realiza búsqueda lineal O(n).

Lista Circular Doblemente Enlazada — `gestion_datos/lista_circular_doblement`

Almacena el historial de proveedores y sus entregas. El último nodo apunta de vuelta al primero tanto en `siguiente` como en `anterior`, formando un ciclo completo.

Cada nodo de proveedor contiene: `nit`, `nombre`, `telefono`, `direccion`, `fecha_entrega`, `numero_factura` y `entregas` (arrayref con los ítems entregados).

Matriz Dispersa — `gestion_datos/matriz_dispersa.pm`

Relaciona **proveedores** (filas) con **fabricantes** (columnas). Solo almacena las celdas con valores distintos de cero usando un hash doble: `$matriz->celdas$i$j = {cantidad, precio}`.

Métodos principales: `nuevo($nf, $nc)`, `insertar($i, $j, $precio, $cantidad)`, `obtener($i, $j)`, `obtener_dimensiones()`, `nombre_filas(@nombres)`, `nombre_columnas(@nombres)`.

Módulo de Validación JSON

`utilidades/validar_json.pm`

> Proporciona tres subrutinas públicas para validar y normalizar los datos antes de insertarlos en las estructuras. Toda la lógica de validación de campos individuales reside en este módulo.

`normalizar_utf8($texto)`

Detecta y decodifica el encoding del texto (UTF-8, ISO-8859-1, cp1252). El `return` dentro del bucle se realiza fuera del bloque `eval` para que retorne correctamente de la subrutina (corrección del bug clásico de `return` dentro de `eval`).

`limpiar_campo($valor, $campo, $tipo)`

Normaliza y valida un campo individual. Tipos soportados:

Tipo	Validación aplicada
`cantidad`	Solo dígitos (`/\d+/`), convierte a número
`nivel_minimo`	Igual que cantidad
`precio`	Dígitos con punto decimal opcional
`fecha`	Patrón uniforme `AAAA-MM-DD` o `AAAA/MM/DD`; validación real con `Time::Piece`
`codigo`	Formato `/^[A-Z]{3,4}-\d+$/i`, retorna en mayúsculas
`tipo_producto`	Solo `MEDICAMENTO`, `EQUIPO` o `SUMINISTRO`
`texto`	Trim de espacios, retorna `undef` si vacío

`validar_campos_obligatorios($item, $campos)`

Detecta campos faltantes incluyendo arrayrefs y hashrefs vacíos (`[]` y `{}`).

Pipeline de Carga Masiva

utilidades/carga_masiva_inventario.pm

Implementa el pipeline completo: lectura del archivo, decodificación UTF-8, parseo JSON, validación campo a campo, enrutamiento por tipo de producto e inserción en la estructura correspondiente.

Flujo de `procesar_archivo($ruta, $ctx)`

1. `_leer_json_con_encoding($ruta)`: Lee bytes crudos, elimina BOM, decodifica con `Encode::decode('U ...)`, parsea con `JSON::PP->new->allow_nonref->relaxed->decode($texto)`. **Nota:** se usa `->decode()` sin flag `->utf8()` porque `$texto` ya es un string Perl interno.
2. `_validar_proveedor($prov)`: Valida NIT y nombre (obligatorios) y campos opcionales.
3. Por cada ítem en `entrega`: llama a `_validar_item($item)`.
4. Según `$tipo`: inserta en `$ctx->med`, `$ctx->bst` o `$ctx->btree`.
5. Si el proveedor tiene entregas válidas, se registra en `$ctx->prov`.
6. `_guardar_estructuras($ctx)`: persiste los cuatro archivos JSON.

utilidades/carga_masiva_usuarios.pm

Sigue el mismo patrón de decodificación. La validación adicional en `_validar_usuario` verifica:

- Formato de colegio `COL-XXXXX` (regex `/^[A-Z]{3,4}-\d+$/i`).
- Tipo de usuario en `{TIPO-01 .. TIPO-05}`.
- Departamento en `{DEP-ADM, DEP-MED, DEP-CIR, DEP-LAB, DEP-FAR}`.
- Contraseña con longitud ≥ 4 caracteres.
- No existencia previa en el AVL (`$avl->buscar($colegio)`).

Interfaz Gráfica GTK3

Arquitectura

La interfaz está fragmentada en módulos por responsabilidad única. Cada tab del `Gtk3::Notebook` principal es responsabilidad de un módulo independiente bajo `interfaz/admin/`. El estado compartido se pasa como hashref `%ctx`.

Contexto compartido (`%ctx`)

`interfaz/admin/common.pm`

Módulo de helpers sin estado. Expone: `boton($texto, $clase)`, `campo_entry($texto, $es_contra)`, `tab_label($texto)`, `msg_info($msg)`, `confirmar_y_ejecutar($msg, $cb)`, `crear_vista_generica($st headers)`, `mensaje($lbl, $txt, $error)`.

`interfaz/admin/personal_medico.pm`

Implementa el sistema de filtros dinámicos con `Gtk3::TreeModelFilter`. Las variables de filtro `$f_dpto`, `$f_tipo`, `$f_esp` son closures compartidas entre el `set_visible_func` del filter y los `signal_connect` de los botones de filtro dentro del mismo scope de `crear_tab`. Este diseño es crítico: si los widgets de filtro se construyen en una subrutina separada, las variables de closure quedan en scopes distintos y los filtros no funcionan.

Listing 1: Pattern de closure para filtros dinámicos

```
sub crear_tab {
    my ($ctx) = @_;
    my $avl   = $ctx->{avl};

    # Store + filter en el mismo scope
    my $store  = Gtk3::ListStore->new(...);
    my $filter = Gtk3::TreeModelFilter->new($store, undef);

    # Variables compartidas por closure
    my ($f_dpto, $f_tipo, $f_esp) = ('', '', '');

    $filter->set_visible_func(sub {
        my ($model, $iter) = @_;
        # usa $f_dpto, $f_tipo, $f_esp del scope externo
        return FALSE if $f_dpto && $model->get_value($iter, 3) ne
           $f_dpto;
        return TRUE;
    });

    $btn_filt->signal_connect(clicked => sub {
        # modifica las mismas variables de closure
        $f_dpto = ...; $filter->refilter();
    });
}
```

Generación de Reportes Graphviz

Flujo general

1. `reportes::nombre_archivos::nombre($tipo, $prefijo)` genera rutas únicas con timestamp: `reportes/AVL_20260323_143022.{dot,png,svg}`.
2. El módulo de reporte correspondiente escribe el archivo `.dot`.
3. Se ejecuta: `system("dot -Tpng $dot -o $png")`.
4. La interfaz carga el PNG con `Gtk3::Gdk::Pixbuf->new_from_file($png)` y lo muestra en un `Gtk3::Image` dentro de un `ScrolledWindow`.

Características por reporte

Módulo	Características del diagrama
`avl_dots.pm`	Nodos circulares con colegio, nombre, tipo, departamento; flechas L/R diferenciadas
`bst_dots.pm`	Nodos rectangulares con código, nombre y cantidad; indicación visual de raíz y hojas
`btree_dots.pm`	Nodos como bloques segmentados horizontales (una celda por clave); amarillo = lleno (3 claves), verde = con espacio
`matriz_dots.pm`	Cabeceras de fila (proveedores) y columna (fabricantes) rectangulares; nodos de valor circulares con la cantidad total
`inventario_dots.pm`	Lista doble con nodos doblemente enlazados de medicamentos
`proveedores_dots.pm`	Lista circular doble de proveedores

Persistencia JSON

utilidades/leer_json.pm

Dos funciones principales:

Listing 2: Guardar y cargar JSON

```
# Guardar un arrayref como JSON formateado
sub guardar_json {
    my ($ruta, $datos) = @_;
    my $json = JSON::PP->new->utf8->pretty->canonical(1);
    open my $fh, '>:encoding(UTF-8)', $ruta or die "No se pudo escribir: $!";
    print $fh $json->encode($datos);
    close $fh;
}

# Cargar JSON; retorna arrayref vacio si el archivo no existe o es invalido
sub parsear_json_posiblemente_vacio {
    my ($ruta) = @_;
    return [] unless -e $ruta;
    open my $fh, '<:encoding(UTF-8)', $ruta or return [];
    local $/;
```

```
    my $texto = <$fh>;
    close $fh;
    my $json = JSON::PP->new->allow_nonref->relaxed;
    my $datos;
    eval { $datos = $json->decode($texto) };
    return $@ ? [] : (ref($datos) eq 'ARRAY' ? $datos : [$datos]);
}
```

Regla de decodificación: cuando se abre el archivo con `<:encoding(UTF-8)`, Perl decodifica los bytes a string interno. En ese caso se debe usar `JSON::PP->decode()` **sin** la flag `->utf8()`, que esperaría bytes crudos y causaría doble decodificación.

Módulo de Solicitudes y Pedidos

interfaz/admin/solicitudes.pm

El módulo gestiona el flujo completo de solicitudes de reabastecimiento:

Función	Responsabilidad
`_obtener_todas_solicitudes()`	Lee `datos/solicitudes/todas`
`_cambiar_estado_solicitud($uuid, $estado)`	Actualiza el estado en el archivo global y en el archivo por departamento
`_generar_pedido_proveedor($codigo, $cantidad, $dep)`	Crea un pedido en `datos/pedidos.json`
`_generar_pedido_directo($prov, $cod, $nombre, $cant, $razon)`	Pedido manual del administrador
`_marcar_pedido_entregado($prov, $med, $cant)`	Cambia estado a Entregado
`_actualizar_inventario($codigo, $cantidad)`	Incrementa la cantidad del medicamento

Estructura de archivos de solicitudes

```
datos/solicitudes/
          todas_solicitudes.json     <- Todas las solicitudes del
   sistema
          DEP-MED/
                solicitudes.json       <- Solicitudes del departamento
   MED
          DEP-CIR/
                solicitudes.json
          ...
```

Cada solicitud contiene: `uuid`, `fecha_solicitud`, `departamento`, `solicitante`, `codigo_medicamento`, `cantidad`, `prioridad` (Normal/Alta/Urgente), `razon`, `estado` (Pendiente/Aprobado/Rechazado).

Sistema de Estilos GTK3

interfaz/estilos.pm

El CSS se carga mediante `Gtk3::CssProvider` con prioridad `STYLE_PROVIDER_PRIORITY_APPLICATION`, sobrescribiendo el tema del sistema operativo para los widgets de la aplicación.

Widgets y selectores críticos

Widget	Selector CSS
ComboBoxText	`combobox button, combobox button.combo`
SpinButton	`spinbutton`, `spinbutton entry`, `spinbutton button`
TextView (logs)	`textview`, `textview text`
Entry deshabilitado	`entry:disabled`
Notebook tab activo	`notebook tab:checked`

Los `ComboBoxText` y `SpinButton` en GTK3 heredan el tema oscuro del sistema operativo si no se estilan explícitamente. En el CSS es necesario definir el selector `combobox button.combo` para que el botón desplegable muestre fondo claro y texto visible.

Manejo de Errores

El sistema implementa manejo de errores en tres capas:

Capa de validación (`validar_json.pm`): retorna `undef` para campos inválidos. Los módulos de carga masiva acumulan advertencias en un array `@advertencias` y las reportan en el resumen final sin interrumpir el procesamiento de ítems válidos.

Capa de I/O (`leer_json.pm`): los `open` usan `or die` en escritura y `or return []` en lectura. El parseo JSON se envuelve en `eval { ... }` para capturar JSON inválido sin terminar el programa.

Capa de interfaz (`solicitudes.pm` y módulos de tab): los callbacks de botones usan `eval { ... }` para capturar errores de persistencia y mostrarlos en labels de mensaje dentro de la UI en lugar de terminar la aplicación.

Conclusiones

- La fragmentación en módulos por responsabilidad (cada tab = un módulo) facilita el mantenimiento y permite modificar una funcionalidad sin afectar las demás.
- El patrón de closure para filtros dinámicos en `personal_medico.pm` es la solución idiomática en Perl/GTK para compartir estado entre callbacks sin recurrir a variables globales.
- La regla de decodificación UTF-8 (usar `->decode()` sin `->utf8()` cuando el texto ya está decodificado) previene el error de «malformed UTF-8» que afectaba a la carga masiva.
- La generación de reportes Graphviz dentro de la propia interfaz (sin abrir ventana externa) mejora la experiencia del usuario y cumple el requerimiento de visibilidad de reportes establecido en el enunciado.
- Las cinco estructuras de datos implementadas desde cero demuestran el dominio de los algoritmos de balanceo (AVL), búsqueda binaria (BST), división de nodos multicamino (Árbol B), listas enlazadas bidireccionales y matrices dispersas.